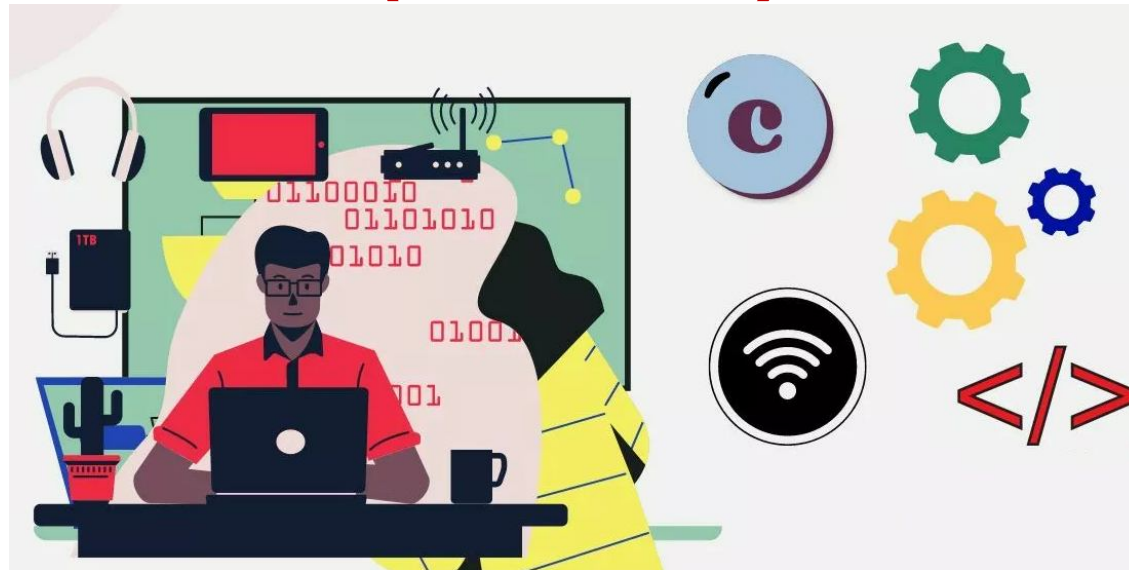




# ***Structured Programming Concepts***

***( 26CSE0105)***



## ***Introduction to C Programming***

**Prepared By: Dr. Punit Soni**

**Department of Computer Science and Engineering**

**Chitkara University Institute of Engineering and Technology**

**Chitkara University, Punjab**

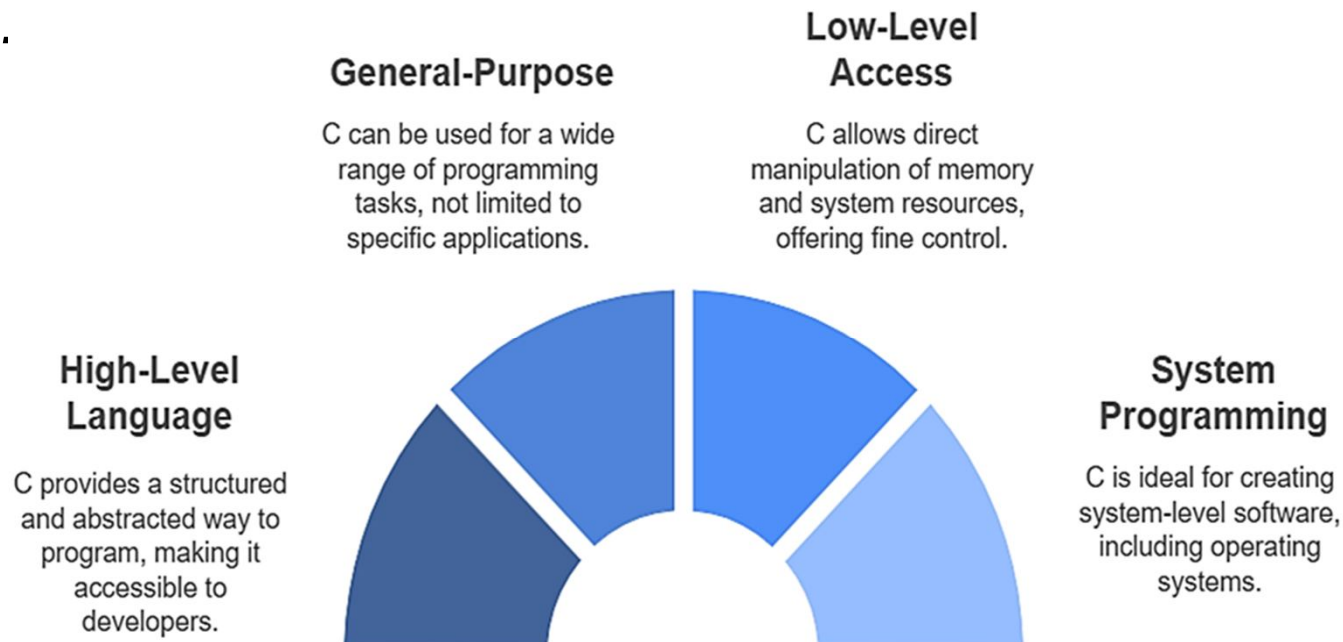
# Topics to be covered

- ***Introduction to C Programming***
- ***History of C Programming***
- ***Features of C Programming***
- ***Structure of a C Program***
- ***Compilation and Execution Process***

# Introduction to 'C' Language

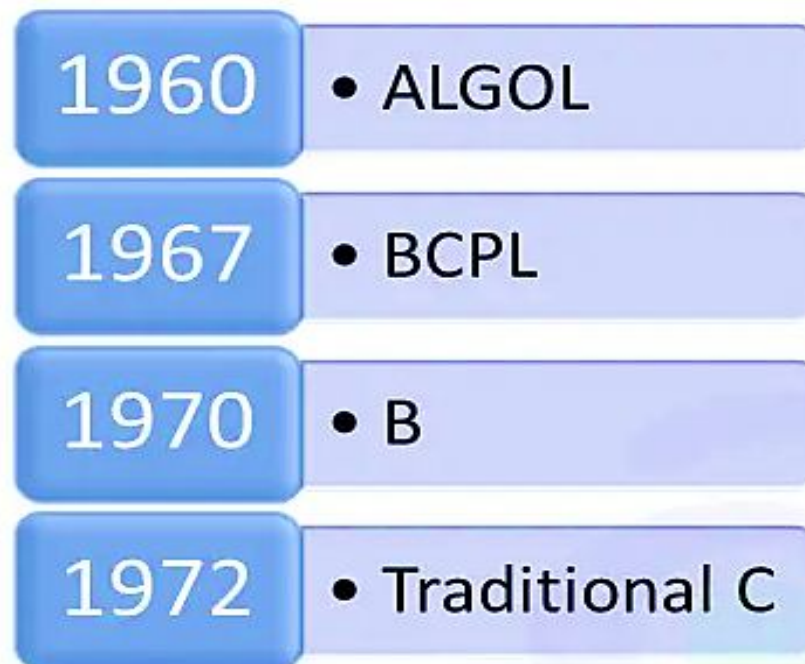


- The **'C' language** is a **general-purpose** and **procedural-oriented** programming language.
- It is a **structured** and **machine-independent** programming language.
  - Developed by **Dennis Ritchie** in **1972**.
  - In **Bell Laboratories** of **AT&T Labs**.
- **'C'** was mainly developed to **re-implement** the **Unix operating system**.

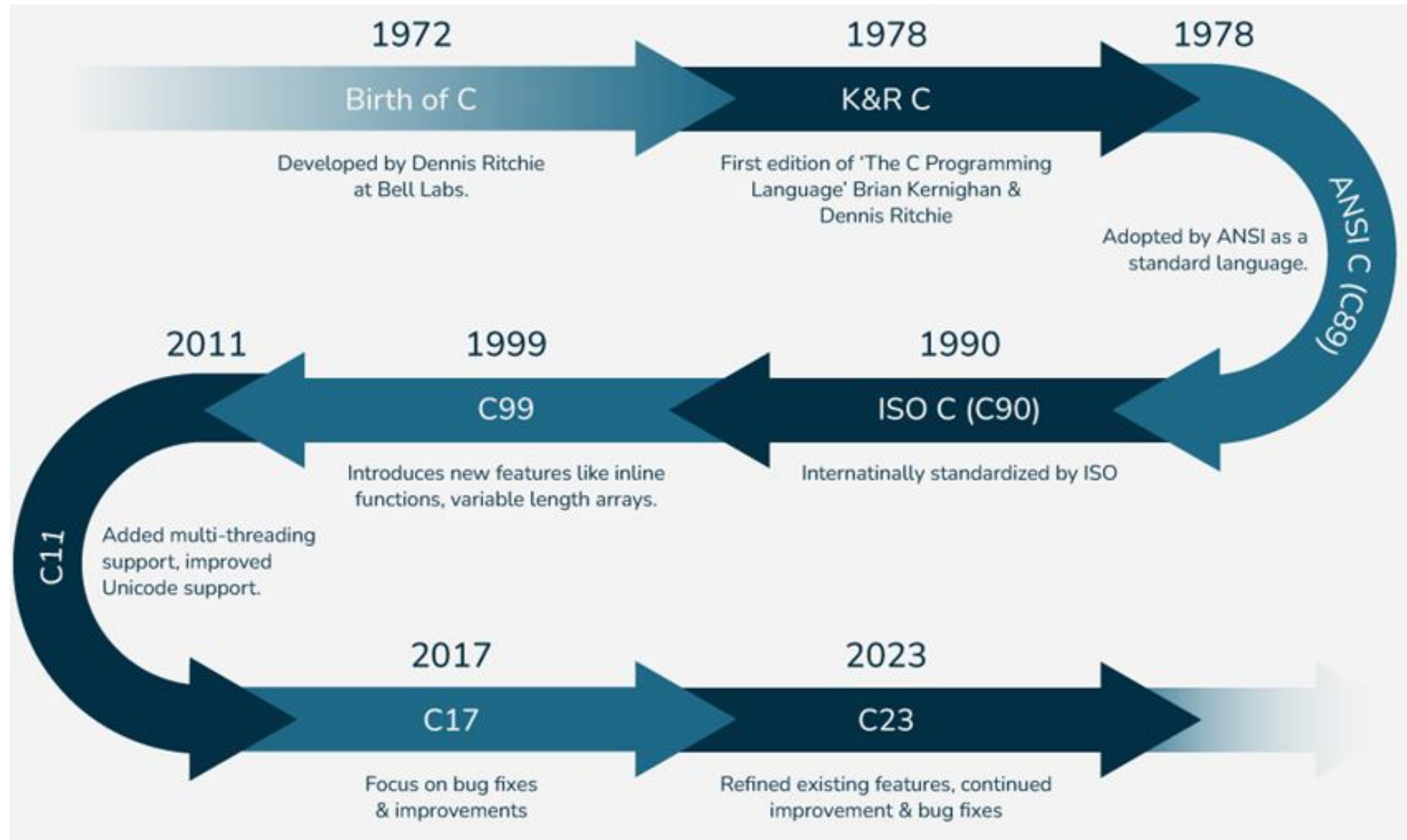


# History of 'C' Language

- The root of all modern languages is **ALGOL** (**Algorithmic Language**).
- '**C**' was developed to **overcome** the **limitations** of **B** and **BCPL** languages.
- These languages had **no real data typing capability**, and their **structured programming** was on the primitive side.



# Timeline of 'C' Language



# Features of 'C' Language

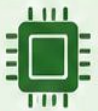


# Role of 'C' in System Programming



## 1. SYSTEM PROGRAMMING

- **System programming** involves **developing software** that manages **hardware**.
  - It provides a **platform** for **application programs** to run.
  - Like **OS**, **Compilers**, and **Device drivers**.



## 2. LOW-LEVEL ACCESS TO MEMORY

- '**C**' provides **low-level access** to memory.
  - Supports **direct manipulation** of hardware resources.
  - Use **pointers** for memory management.



## 3. INTERACTION WITH HARDWARE & SYSTEM RESOURCES

- The '**C**' language **enables interaction** with hardware and system resources.



## 4. WIDELY USED IN SYSTEMS / SOFTWARE DEVELOPMENT

- Most **operating systems/softwares** are written in '**C**'. For example;
  - **UNIX**, **Linux kernels**, **Windows** and **Symbian**
  - **Compilers** and **Interpreters**
  - **Embedded systems**
  - **Firmware**, **Device drivers**, **System utilities**



UNIX /  
Linux



Windows



Symbian



Compilers &  
Interpreters



Embedded  
Systems



Firmware, Device  
Drivers, System  
Utilities

# 'C' as foundation of other languages

- '**C**' is known as the **base** or **mother** of many modern PL's.
- It introduced **core programming concepts** still used today.
- Learning '**C**' helps in understanding **how other languages** work.

- **Core Concepts introduced by 'C':**

- **Structured programming** using **functions**
- Use of **variables** and **data types**
- **Control statements** like **if, switch, loops**
- **Pointers** and **memory management** concepts

# 'C' as foundation of other languages

- Languages Influenced by 'C';

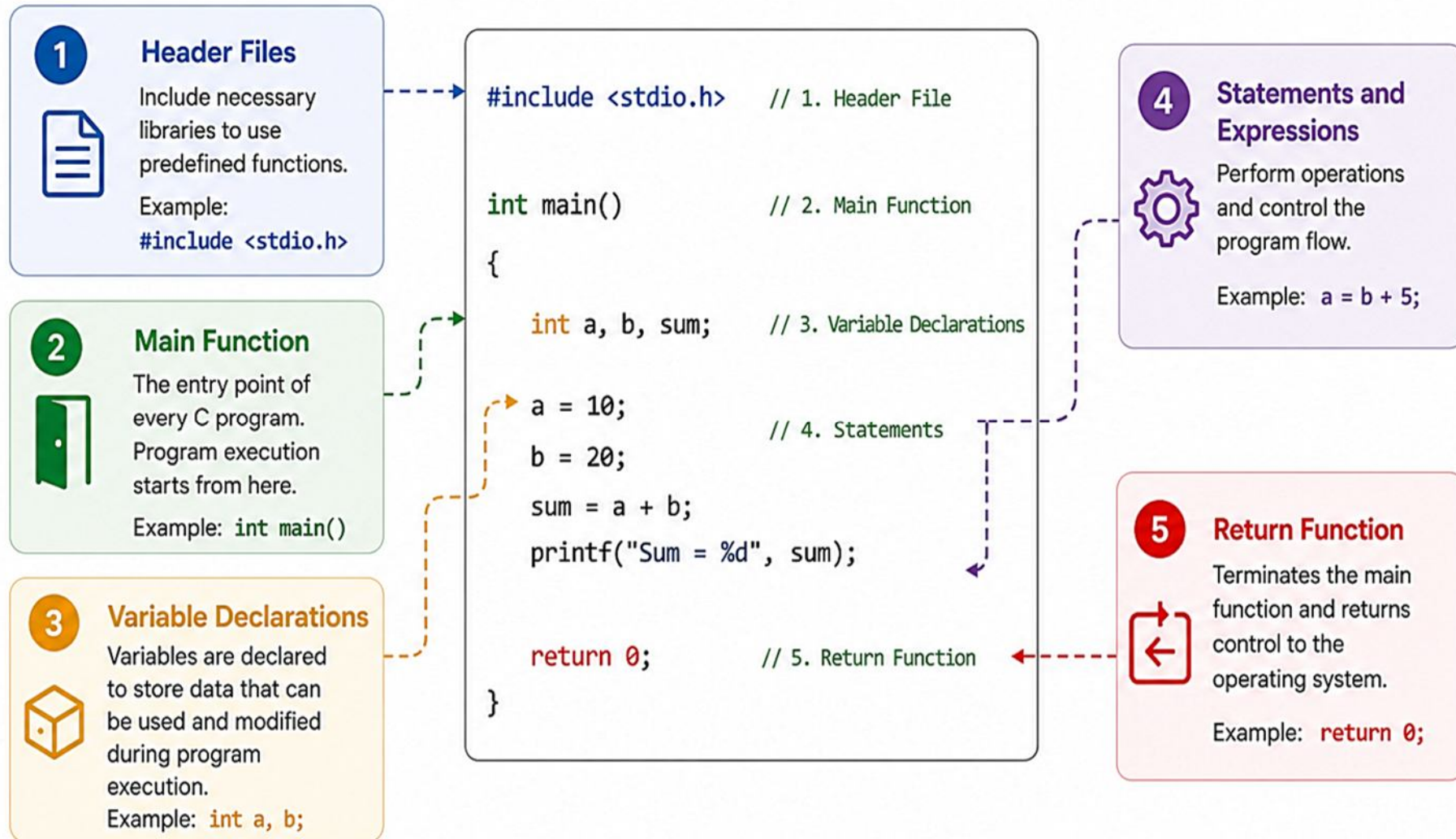
- **C++** → *object-oriented extension of C*
- **Java** → *syntax and control structures inspired by C*
- **C#** → *similar syntax and structure to C*
- **Python & JavaScript** → *influenced by C-style syntax*

# Basic Syntax/Structure of C Program

- The **basic syntax** of C programming involves **fundamental rules** for **structure** and **building blocks**.

|      |   |                                       |           |
|------|---|---------------------------------------|-----------|
|      | 1 | <code>#include &lt;stdio.h&gt;</code> | Header    |
|      | 2 | <code>int main(void)</code>           | Main      |
| BODY | 3 | <code>{</code>                        |           |
|      | 4 | <code>printf("Hello World");</code>   | Statement |
|      | 5 | <code>return 0;</code>                | Return    |
|      | 6 | <code>}</code>                        |           |

# Basic Syntax/Structure of C Program



# Preprocessor Directives

- **Preprocessor directives** are instructions given to the **compiler**.
  - They are processed **before actual compilation starts**.
  - All **preprocessor directives** begin with the symbol (#).
  - They **do not end** with a **semicolon (;)**.

❖ **#include** – for including **header files**

❖ **#define** – for **macro definitions**

❖ **#undef** – for **undefining macros**

❖ **#if, #ifdef, #ifndef** – for **conditional compilation**

# What is main() ?

- The **main()** function always returns an integer value, hence there is an **int** before **main()**, returning **zero** (**0**), indicates **success**.
- If for any reason the statements in **main()** fail to do their intended work we can return a **non-zero** number, indicates **failure**.
- Some compilers like **Turbo C/C++** even permit us to return **nothing** from **main()**.
- In such a case we use keyword **void**.
- But this is non-standard way of writing the **main()** function.

# What is main() ?

- **main()** is a function.
- A **function** is nothing but a **container** for a **set of statements**.
- In a C program there **can be multiple functions**.
- All statements that belong to **main()** are enclosed within a pair of **braces {}**.

```
int main( )  
{  
    statement 1 ;  
    statement 2 ;  
    statement 3 ;  
}
```

# Form of a C program (rules)

- *Each instruction in a C program is written as a **separate statement**.*
- *All statements should be in **lower case letters**.*
- *The statements in a program **must appear** in the **same order** in which we wish them to be executed.*
- *Every C statement must end with a **semicolon** ( ; ). i.e.; **statement terminator**.*

# Comments in C

Few things **need to remember** while writing comments in **C**:

- Comment should be enclosed within `/* */`.  
`/* formula for simple interest */  
si = p * n * r / 100 ;`
- Comments cannot be nested.

```
/* Cal of SI /* Author: gekay date: 25/06/26 */ */
```

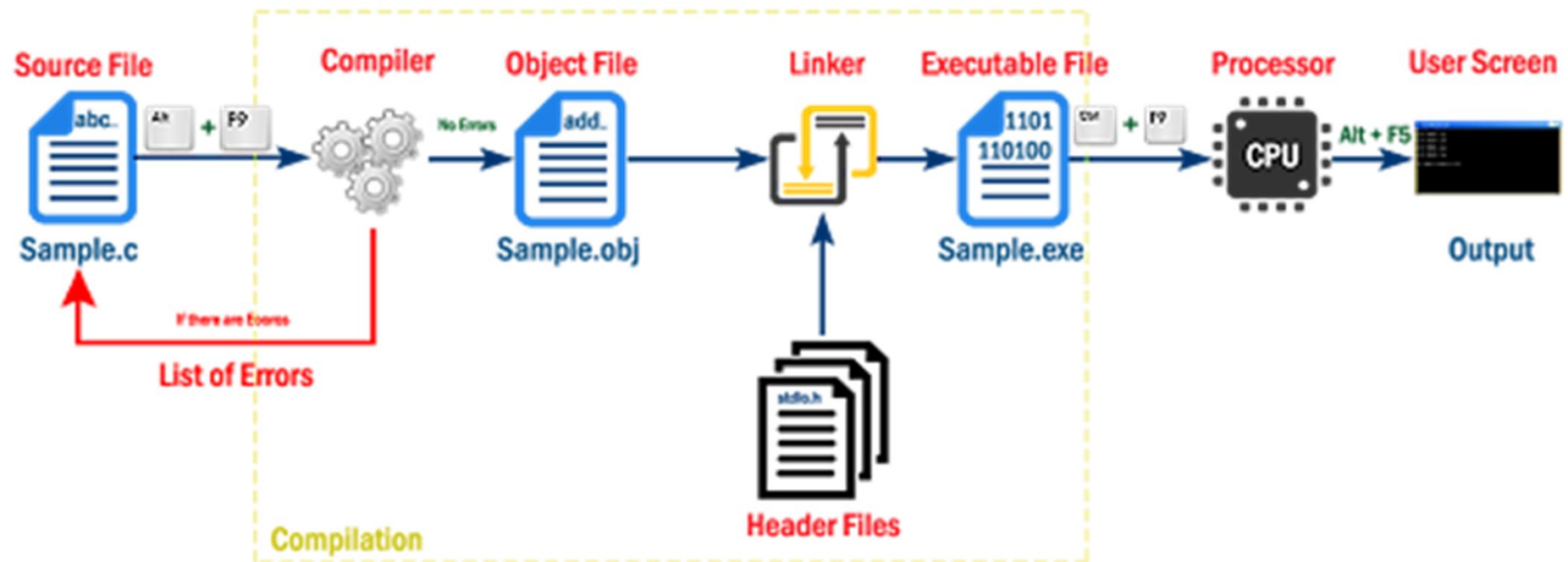
- A comment can be split over more than one line.

```
/* This comment has  
three lines  
in it */
```

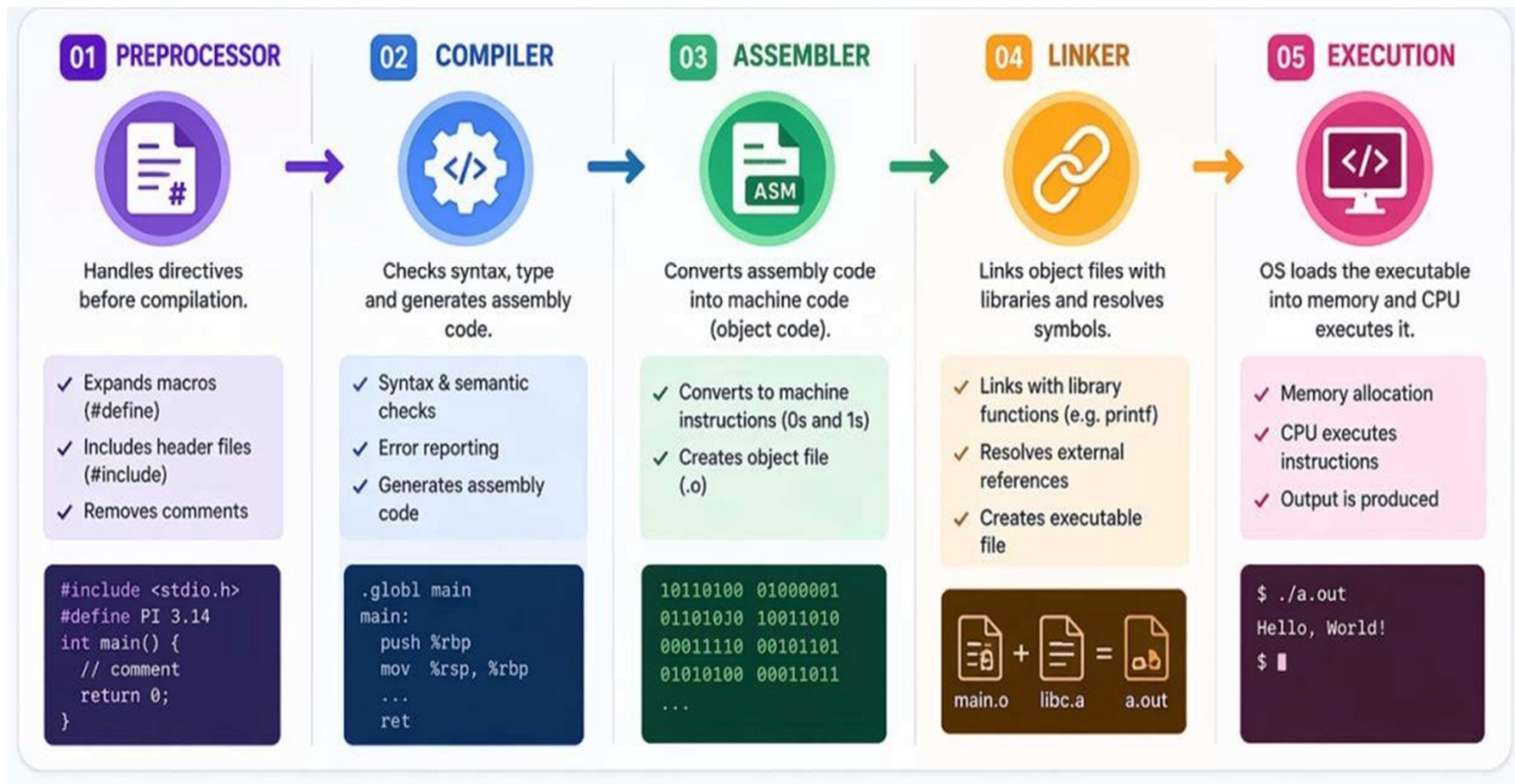
- Single-line comments can also start with `//`.

```
// Calculation of simple interest  
// Formula
```

# Compilation and Execution Process



# Compilation and Execution Process





Thank You!

